# *Building a Smarter Internet*

*David A. Smith*

*YZ.social*

*Neurons that fire together, wire together.*
Donald Hebb · 1949

*An N-DHT Explainer v0.3.48 2026-05-05*

## *I The Problem*

Imagine you and a million strangers each hold one piece of a giant jigsaw puzzle. Someone walks up and asks for piece #438,291. How do you find it?

**Option 1: Have a central directory.** Some company keeps a giant list: "piece 438,291 is held by Alice." Easy to look up—but now that company controls everything. They can shut you down, spy on you, charge you money, or just go bankrupt and take the directory with them.

**Option 2: Give every piece an "address," and design a system where the network *itself* knows how to route requests to the right person, with no central directory.** This is what a **distributed hash table** (DHT) does.[1]

The trick: how do you find the closest node when you don't know who's in the network and you only know a handful of peers?

The dominant DHT algorithm is called **Kademlia**, designed in 2002. Every node has a 160-bit ID—a very large random number. To measure "distance" between two IDs, you XOR them, comparing bit by bit. To find a target, you ask the closest peer you know. They tell you about peers *they* know that are even closer. You ask one of those. Repeat.

The math is clean: each step at least *halves* the remaining distance, so you reach any target in about $\log_2(N)$ hops. With a million nodes, that's 20 steps. Provably correct, used in production all over the internet.

There's just one problem.

[1] DHTs are the machinery behind BitTorrent, IPFS, parts of Ethereum, and the hidden services on Tor. Every node gets a random ID; every piece of content gets an ID; lookups route from one to the other.

*XOR* is a bitwise operation that returns 1 wherever two bits differ and 0 wherever they match. Two IDs that share many leading bits XOR to a small number; two unrelated IDs XOR to a large one.

## *II The Latency Tax*

Twenty hops sounds fast, but each hop sends a message between two random computers somewhere on Earth. If your peers are scattered randomly across the globe, the average pair sits about half the planet apart—roughly 100 milliseconds one-way.

20 hops × 100 ms = **2 seconds.** To find something. Every time.

For real-time applications—voice calls, online games, live messaging, push notifications—2 seconds is unusable. The math says routing is logarithmic, which is fast; the physics says each step is slow because peers are far apart. That's the tension.

This paper attacks the latency problem with two ideas, one of which is borrowed straight from neuroscience.

## *III Putting the Address in the Address*

The first idea is almost embarrassingly simple. It's called **G-DHT** (Geographic DHT).

Kademlia node IDs are random. A node in Tokyo and a node in Berlin have IDs with no relationship to where they actually are. That's *why* hops are random and slow—random IDs mean random geography.

G-DHT changes one thing: the **first 8 bits** of every node ID encode where the node says it is on Earth.

The trick uses Google's S2 library, which divides Earth's surface into cells along a curve called a **Hilbert curve**. Its useful property: places near each other on Earth get cell numbers near each other. So XORing two S2 cell numbers gives a small result for nearby places and a large one for distant places.

Hilbert curves are space-filling: they trace a single continuous path that visits every cell in a grid. Their useful property is that consecutive points on the curve are also adjacent in space.

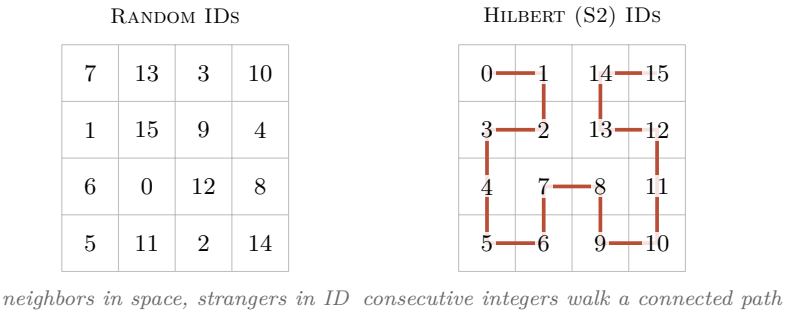


Figure 1: Two arrangements of the integers 0–15 on a 4 × 4 grid. With random IDs, sequential numbers scatter unpredictably; cells that are spatial neighbors are numerical strangers. With S2's Hilbert-curve IDs, sequential numbers form a connected path—a continuous walk that never leaves a small neighborhood. XOR distance in ID space now approximates distance on the ground.

Now your node ID looks like: `[8-bit geographic cell][56-bit hash of public key]`.

The routing algorithm doesn't change at all, but XOR distance now approximately tracks physical distance. When a node looks for a "close" peer in ID space, it tends to find one that's also physically

close. Local traffic stays local. The 20-hop world tour becomes a 13-hop journey around the neighborhood, with each hop maybe 7 ms instead of 100.

Total: about 91 ms instead of 2 seconds. **Roughly 20× faster** for regional traffic, just from changing the *structure* of the addresses.

The cost: nodes can lie about where they are. This is a "cooperative trust" assumption. Mitigations exist, but for now G-DHT trades some defense against location-spoofing for a large speedup.

An attacker who falsifies their geographic prefix degrades the locality benefit but not routing correctness. Mitigations exist—learned coordinates, geographic proof-of-work, IP–ASN binding—but G-DHT trades some defense against location-spoofing for a large speedup.

## *IV A Network That Learns Like a Brain*

The second idea, **NH-1**, is the heart of the paper. It asks a different question:

> *What if, instead of engineering shortcuts into the network, the network learned its own shortcuts based on which paths actually work?*

To explain how, we reach for a strange-but-perfect analogy: the human brain.

### *The Engineering Problem That Brains Already Solved*

A single neuron in your brain connects to roughly 10,000 others through structures called **synapses**. There are about 86 billion neurons total. Any given neuron *could* connect to almost any other, but maintaining synapses is energetically expensive—so neurons are stuck with a hard cap, surrounded by far more potential partners than they can afford to keep.

How does the brain decide *which* connections to keep?

This is the same problem facing a peer in a peer-to-peer network. A web browser can only maintain about 50–100 simultaneous WebRTC connections. The network might have hundreds of thousands of nodes. Which 50 do you keep?

WebRTC is the technology that lets browsers talk directly to each other, without a server in the middle. Safari caps at about 95 simultaneous connections; Firefox at about 190.

The brain solved this problem hundreds of millions of years ago. The solution is called **long-term potentiation**, or LTP.

### *The Brain's Trick*

In 1973, two scientists—Bliss and Lømo—ran a now-classic experiment. They zapped a particular pathway in a rabbit's brain with high-frequency electrical pulses. Afterward, that pathway responded *more strongly* to subsequent signals—not for seconds, but for *weeks*. The connections had physically gotten stronger.

Decades of follow-up research filled in the details:

1. **The coincidence detector.** Synapses have a special kind of receptor (called NMDA) that only activates when *both* sides of the connection fire at roughly the same time. It's not "I fired" or "you fired"—it's specifically "we fired together."

2. **The strengthening signal.** When that coincidence happens, calcium rushes in and triggers the synapse to insert *more* of a different kind of receptor (AMPA), making the connection physically more sensitive.

3. **Consolidation.** Over the next half hour or so, new proteins get made and new physical structures grow. The change becomes permanent.

4. **Decay.** Connections that *don't* get used regularly weaken over time, in a process called long-term depression (LTD).

5. **A protective tag.** Recently strengthened synapses get a temporary "tag" that protects them from being overwritten by competing changes—but only for a brief window.

Without this tag, learning would be self-destructive: the synapses you just learned were useful would be the *first* ones overwritten by the next signal.

Donald Hebb summarized this in 1949—before the molecular details were known—in the rule that's now bedrock in neuroscience and AI:

> *Neurons that fire together, wire together.*

Every artificial neural network you've ever heard of—every part of ChatGPT, every image recognition system, every recommendation algorithm—ultimately runs on a digital descendant of this rule.

### *Translating Neurons into Network Routing*

The claim is that the brain's mechanism translates *literally* to peer-to-peer routing—no metaphor needed:

| In the brain | In NH-1 |
|---|---|
| A synapse (neural connection) | A connection to a peer |
| Synaptic strength (weight 0 to 1) | A learned weight on each connection |
| LTP: co-firing strengthens | A successful lookup increments the weights along its path |
| LTD: disuse weakens | Every connection slowly decays over time |
| Synaptic tagging (protection) | Recently used connections are protected briefly |
| Pruning of unused connections | Lowest-scoring connection is evicted when a new one wants in |

That is what NH-1 *is*: a routing system where every connection has a weight that goes up when used and decays when not. The network's "memory" is its routing table, and the routing table evolves into whatever shape best serves the actual traffic.

### *The Vitality Function*

The core of NH-1 is a single equation that scores every connection:

$$vitality \;=\; weight \;\times\; recency$$

The **weight** is a number between 0 and 1, updated like this:

- Every successful lookup that uses the connection raises its weight by 0.05 (capped at 1).
- Every "tick" of the clock multiplies every weight by 0.995—a slow decay.

The **recency** is 1.0 for the first 20 ticks after a connection is used—the protective tag from synaptic tagging—then drops off exponentially.

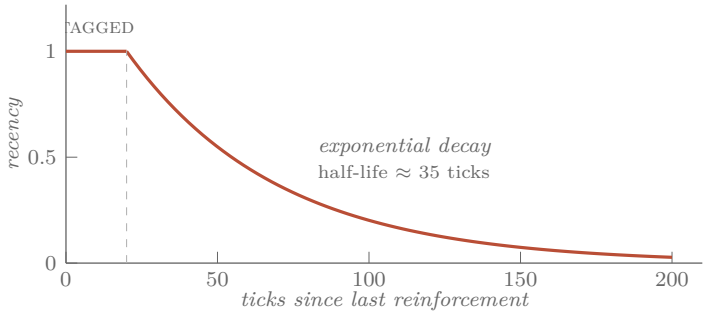


Figure 2: The recency multiplier in NH-1's vitality function. For 20 ticks after a connection is used, recency stays locked at 1.0—the synaptic tag, protecting freshly learned shortcuts from being immediately overwritten. After that, it decays exponentially with a half-life of about 35 ticks. A connection unused for ∼150 ticks contributes essentially nothing to its vitality, regardless of accumulated weight.

When a new connection wants in but the routing table is full, the connection with the lowest vitality gets evicted. Frequently used connections accumulate weight and stay. Unused ones decay and get evicted. Recently strengthened ones are temporarily protected.

That's it. That's the core.

### *Why the Consolidation Matters*

The previous version of this system (called NX-17) had **18 different rules and 44 parameters** controlling things like "how full should each part of the routing table be?" and "when should we evict a peer?" and "how much should we prefer nearby peers?"

NH-1 replaces all of that with the single vitality function plus **12 rules and 12 parameters**. Letting the connections themselves learn through reinforcement collapses a tangle of rules into one principle.

## *V The Five Operations*

EVERY BEHAVIOR in NH-1 falls into one of five categories—and the categories mirror how *any* adaptive system works:

**1. Navigate** — pick the next hop for a lookup. NH-1 scores each candidate by combining XOR distance progress, learned weight, and observed latency. The latency factor halves the score every 100 ms—so a peer that's mathematically a *bit* further but physically a *lot* closer wins.

**2. Learn** — strengthen what works. Four mechanisms run on every successful lookup:

- **LTP**: every connection on a *fast* successful path (one that beats the running average of recent path latencies) gets stronger.
- **Hop caching**: every intermediate node along the path remembers the *destination*, not just the next hop. Next time, the path is shorter.
- **Triadic closure**: if you keep seeing peer A relay messages to peer C through you, you introduce them directly. The triangle "A–you–C" becomes a direct edge "A–C."

Named after social network theory: dense triangles emerge in any network shaped by interaction.

- **Incoming promotion**: peers who keep reaching out to you become candidates for *your* routing table. Passive learning—the network notices who's interested in you, not just who you're interested in.

**3. Forget** — decay everything that isn't reinforced; evict by vitality.

**4. Explore** — inject occasional randomness. A purely greedy router would lock onto the first decent shortcut and never find better ones. NH-1 has two exploration tricks: a "temperature" that decays over time but spikes when something breaks (heat up when surprised, cool down when stable), and an "epsilon-greedy" rule that picks a random first hop 5% of the time, just to keep options alive.

**5. Structure** — bootstrap and maintain the basic shape of the network when new nodes join.

The template is universal: act, learn from feedback, forget the obsolete, occasionally try something new, maintain basic structure. This is how any good adaptive system has to work.

## *VI Axonal Pub/Sub*

A REAL peer-to-peer network needs more than "find the node holding key X." It also needs **broadcast**: a publisher should be able to send a message to all subscribers of a topic, without knowing who they are.

This is publish/subscribe, or "pub/sub." Think of how YouTube notifies subscribers of a new video—except without YouTube being in the middle.

In neuroscience, the *output* of a neuron—the long branching cable that delivers signals to many downstream targets—is called an **axon**. NH-1 builds axonal delivery trees:

- A topic has an ID, derived from the publisher's location and the topic name.
- Subscribers send a message routed through the DHT toward that topic ID.
- The first node already participating in the topic's tree intercepts the subscriber and adds them as a child.
- When a publisher publishes, the message flows from the root down through all the branches.

That part is standard. The useful property is that **the tree heals itself with no special machinery**. Every member of the tree periodically re-issues its subscribe message. If the tree is intact, nothing changes. If a parent died, the re-subscribe naturally lands on whichever live ancestor is now closest. No heartbeats, no failure detection, no parent tracking. The same mechanism that *builds* the tree *repairs* it.

Result: under 5% churn (5% of nodes joining and leaving constantly), NH-1 still delivers 100% of messages. After three refresh cycles, it recovers from any disruption.

## *VII Hitting the Theoretical Floor*

IN 2004, Frank Dabek and colleagues proved a beautiful, depressing result: **no recursive DHT can be faster than** $3\delta$, where $\delta$ is the median one-way latency between random pairs of nodes on the Internet.[2]

The proof is geometric. Each hop in a DHT covers half the remaining distance. So the total time is $\delta + \delta/2 + \delta/4 + \delta/8 + \cdots$ which converges to $2\delta$. Add one more hop for final delivery and you get $3\delta$. No matter how clever your routing, no matter how big your network, you can't beat this.

[2] Dabek et al., "Designing a DHT for low latency and high throughput," *Proc. NSDI 2004*.

For 20 years, no published DHT had been measured at this floor. The best implementations got to maybe 2× the floor.

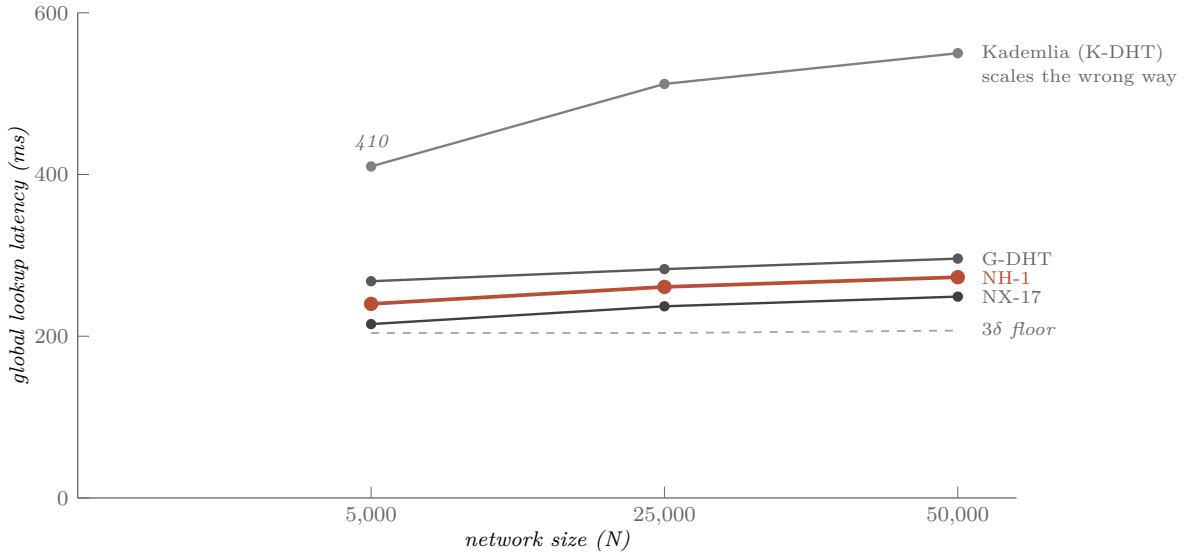


Figure 3: Global lookup latency versus network size, plotted against the Dabek 3$\delta$ analytical floor (gray dashed). Kademlia *worsens* as the network grows—more hops, each at full pairwise latency, each contributing fully to the total. NX-17 hugs the floor at 1.16×; NH-1 at 1.27×. The remaining ∼20% overhead matches what the geometric series predicts for 4–5 hops where an ideal protocol takes 3.

NH-1's predecessor, NX-17, hits 1.16× **the floor** at 25,000 nodes. NH-1 hits 1.27×. They sit at the theoretical limit. The remaining ∼20% overhead is structural—they take about 4 to 5 hops where an ideal protocol would take 3, and each "extra" hop costs about $\delta/2$, exactly as the geometric series predicts.

For comparison, plain Kademlia *gets worse* as the network grows: 2.01× the floor at 5,000 nodes, 2.65× at 50,000. It scales the wrong way.

## *VIII Is It the Learning, or the Geography?*

A SKEPTIC would push back: "Sure, but maybe the geographic prefix is doing all the real work. The brain-inspired learning is gravy."

The paper runs an **ablation study** to answer this. Strip the geographic prefix entirely—random IDs again—and re-run the comparison.

An ablation study removes one feature and re-runs everything to see what that feature actually contributed. It's the cleanest way to attribute a result to a specific mechanism.

Result: with **zero geographic information**, NX-17 still routes **26% faster than Kademlia**, and NH-1 routes **8% faster**. The learning is doing real work, not sharpening pre-existing geographic structure.

This is the kind of clean control experiment that should accompany any claim of this kind.

## *IX Slice World*

The sharpest demonstration is what the paper calls the **Slice World** test. The network gets cut almost in half—Eastern hemisphere on one side, Western on the other, connected only through a *single node* near Hawaii. Every other cross-hemisphere connection is severed.

Can the protocol still route messages between hemispheres?

- **Plain Kademlia: 0% success.** With no learning, the partition is permanent.
- **G-DHT (geography only, no learning): 4.6% success.** The geographic prefix accidentally points a few peers at the bridge, but nothing builds on the discovery.
- **NX-17 and NH-1: ∼94% success.**

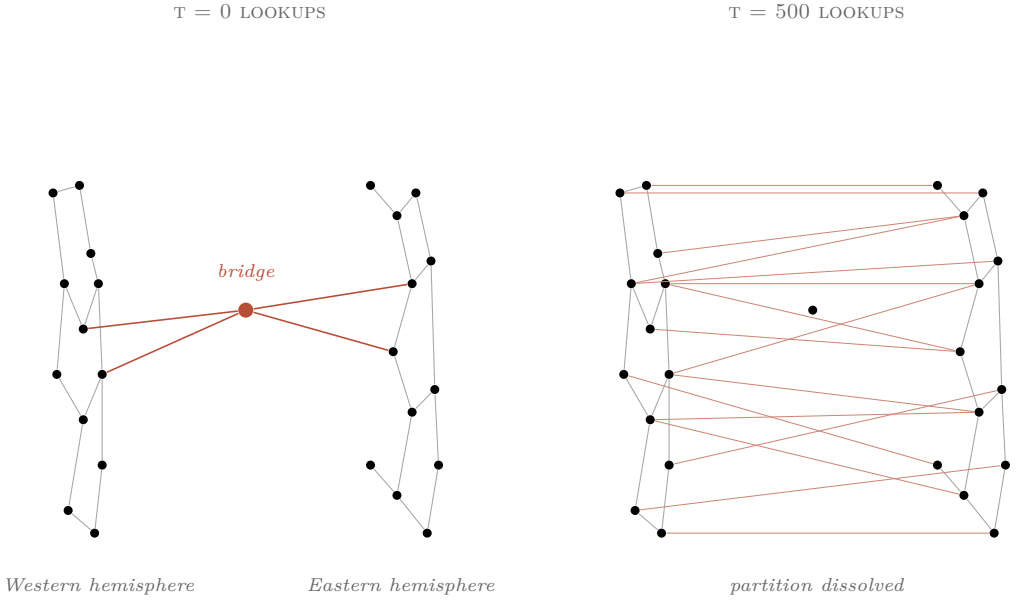


Figure 4: At $t = 0$ (left), the partition has a single bridge node connecting two hemispheres. By $t = 500$ lookups (right), hop caching has installed cross-hemisphere shortcuts in many intermediate nodes; triadic closure has converted observed transit pairs into direct edges. The bridge is no longer special. The protocol does not keep finding the bridge—it uses the bridge to rebuild the bridges that were cut.

The bridge becomes a **seed crystal**. After just 10 lookups through the partition, hop caching has installed cross-hemisphere edges in many intermediate nodes. Triadic closure creates direct connections between peers that keep meeting through the bridge. By 500 lookups, hundreds of cross-hemisphere connections exist. The partition has effectively dissolved.

The protocol doesn't keep finding the bridge—it *uses* the bridge to rebuild the bridges that were cut. Like a brain forming new pathways around damaged tissue.

## X What the Simulator Doesn't Model

The paper is explicit about what *isn't* measured.

The simulator runs about 25,000 lines of JavaScript code modeling the network, but it abstracts away several real-world frictions:

- **Connection setup time.** Real WebRTC connections take 1.5–3 seconds to negotiate. The simulator treats them as instant, so real-world recovery from partitions will be slower than the simulator suggests.
- **Timeout windows.** Real RPCs to dead nodes stall for seconds before failing. The simulator detects death instantly.
- **Bandwidth saturation.** A popular peer will get overloaded. The simulator's latency model doesn't capture this. Real systems risk "success disasters"—everyone routes through the fastest peer until it collapses, then abandons it, then floods back when it recovers, in oscillations that are hard to dampen.
- **Latency jitter.** Real round-trip times vary by ±30% from queuing and congestion. The simulator's latencies are clean and monotone.

These get called out in a "red team" appendix.[3] The paper's results show **the brain working perfectly**; the *body*—actually deploying it on real internet conditions—still needs work.

[3] The companion red-team analysis ranks thirteen deployment-relevant gaps by severity and time-to-fix.

## XI Plasticity of Plasticity

The most interesting future direction is **metaplasticity**—plasticity of plasticity. In real brains, the rules governing learning *themselves* change based on the brain's activity level. A neuron that's been very active becomes harder to strengthen further (otherwise everything saturates). A neuron that's been quiet becomes easier. The learning rules adapt.

NH-1's parameters—decay rate, protection window, exploration rate—are currently hand-picked constants. A metaplastic version would let the network self-tune them based on local conditions. A peer in a stable region would adapt slowly. A peer in a high-churn region would adapt aggressively. The user would set one knob—"I want my lookup failure rate below 1%"—and the network would tune itself to hit it.

That's the next layer of brain-inspired self-organization, and the natural endpoint of the path the paper lays out.

## *XII Why This Matters*

Step back from the technical details. The big idea:

**A peer-to-peer network and a brain face the same engineering problem**—limited connection capacity, vastly more candidates than slots, and the need to figure out which connections are useful.

**The brain's hundreds-of-millions-of-years-old solution maps directly onto network routing.** Strengthen what works. Decay what doesn't. Protect recent learning briefly so it isn't immediately overwritten. Occasionally explore. The result is a network whose structure is shaped by the traffic it carries, rather than by rules an engineer guessed at.

This isn't just a faster DHT. It's a demonstration that adaptive systems with the right learning rules find their own structure—that decades of brain research can collapse a 44-parameter system into a 12-parameter one, and that you can hit a theoretical limit that's stood for 20 years by importing the right idea from a different field.

Code, data, simulator, and the red-team analysis criticizing it are open source.